

UML-диаграммы

Ищенко Алексей Петрович

Система и подсистема

- **Система** - совокупность взаимосвязанных управляемых подсистем, объединенных общей целью функционирования.
- **Подсистема** - это совокупность элементов, часть из которых задает спецификацию поведения других элементов.

Ян Соммервилл объясняет это понятие таким образом:
Подсистема - это система, функционирование которой не зависит от сервисов других подсистем. Программная система структурируется в виде совокупности относительно независимых подсистем.

Модель

В процессе проектирования система рассматривается с **разных точек зрения** с помощью моделей, различные представления которых предстают в форме диаграмм. Опять-таки у читателя могут возникнуть вопросы о смысле понятий *модели* и *диаграммы*. Думаем, красивое, но не слишком понятное *определение модели как семантически замкнутой абстракции системы* вряд ли прояснит ситуацию.

- **Модель** - это некий (материальный или нет) *объект*, отображающий лишь наиболее значимые для данной задачи характеристики системы. Модели бывают разные - материальные и нематериальные, искусственные и естественные, декоративные и математические...

Диаграмма

- **Диаграмма** - это графическое *представление множества* элементов. Обычно изображается в виде графа с вершинами (сущностями) и ребрами (отношениями).

Примеров диаграмм можно привести множество. Это и знакомая нам всем со школьных лет *блок-схема*, и схемы монтажа различного оборудования, которые мы можем видеть в руководствах пользователя, и *дерево* файлов и каталогов на диске, которое мы можем увидеть, выполнив в консоли *Windows* команду `tree`, и многое-многое другое.

В повседневной жизни диаграммы окружают нас со всех сторон, ведь рисунок воспринимается нами легче, чем текст...

Диаграммы

Но вернемся к проектированию *ПО* (и не только)...

В этой отрасли с **помощью диаграмм можно визуализировать систему с различных точек зрения:**

Одна из диаграмм, например, может описывать взаимодействие пользователя с системой, другая - изменение состояний системы в процессе ее работы, третья - взаимодействие между собой элементов системы и т. д. Сложную систему можно и нужно представить в виде набора небольших и почти независимых моделей-диаграмм, причем ни одна из них не является достаточной для описания системы и получения полного представления о ней, поскольку каждая из них фокусируется на каком-то определенном аспекте функционирования системы и выражает разный *уровень абстракции*.

Другими словами, каждая модель соответствует некоторой определенной, частной точке зрения на проектируемую систему.

Диаграммы

- Несмотря на то что ранее мы весьма вольготно обошлись с понятием модели, следует понимать, что в контексте приведенных выше определений **ни одна отдельная диаграмма не является моделью.**
- Диаграммы - лишь средство визуализации модели, и эти два понятия следует различать. Лишь **набор диаграмм составляет модель системы** и наиболее полно ее описывает, но не одна *диаграмма*, вырванная из контекста.

Виды диаграмм

UML 1.5 определял двенадцать типов диаграмм, разделенных на три группы:

- четыре типа диаграмм представляют статическую структуру приложения;
- пять представляют поведенческие аспекты системы;
- три представляют физические аспекты функционирования системы (диаграммы реализации).

Текущая версия *UML* 2.1 внесла не слишком много изменений. Диаграммы слегка изменились внешне (появились фреймы и другие визуальные улучшения), немного усовершенствовалась *нотация*, некоторые диаграммы получили новые наименования.

Виды диаграмм

Впрочем, *точное число канонических диаграмм* для нас абсолютно неважно, так как мы рассмотрим не все из них, а лишь некоторые - *по той причине*, что количество типов диаграмм для конкретной модели конкретного приложения не является строго фиксированным.

Для простых приложений нет необходимости строить все без исключения диаграммы. Например, для локального приложения не обязательно строить диаграмму развертывания.

Важно понимать, что перечень диаграмм зависит от специфики разрабатываемого проекта и определяется самим разработчиком. Если же все-таки хочется узнать обо всех диаграммах *UML*, отправляемся к стандарту *UML*:

http://www.omg.org/technology/documents/modeling_spec_catalog.htm#UML

Виды диаграмм:

- *диаграмма прецедентов;*
- диаграмма классов;
- *диаграмма объектов;*
- диаграмма последовательностей;
- диаграмма взаимодействия;
- диаграмма состояний;
- *диаграмма активности;*
- *диаграмма развертывания.*

Диаграмма прецедентов (use case diagram)

Любые (в том числе и программные) системы проектируются с учетом того, что в процессе своей работы они будут использоваться людьми и/или взаимодействовать с другими системами.

Сущности, с которыми взаимодействует система в процессе своей работы, называются **экторами**, причем каждый **эктор** ожидает, что система будет вести себя строго определенным, предсказуемым образом.

Диаграмма прецедентов (use case diagram)

Попробуем дать более строгое определение эктора. Для этого воспользуемся замечательным визуальным словарем по UML *Zicom Mentor*:

- **Эктор (actor)** - это множество логически связанных ролей, исполняемых при взаимодействии с *прецедентами* или сущностями (система, подсистема или класс). Эктором может быть человек или другая система, подсистема или класс, которые представляют нечто вне сущности.

Диаграмма прецедентов (use case diagram)

Графически эктор изображается либо "человечком", подобным тем, которые мы рисовали в детстве, изображая членов своей семьи, либо *символом класса с соответствующим стереотипом*, как показано на рисунке. Обе формы представления имеют один и тот же смысл и могут использоваться в диаграммах. "Стереотипированная" форма чаще применяется для представления системных экторов или в случаях, когда эктор имеет свойства и их нужно отобразить

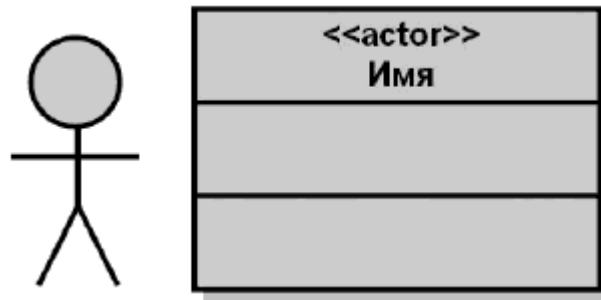


Диаграмма прецедентов (use case diagram)

- Можно задать вопрос: *а почему эктор, а не актер?* Причина же, почему мы говорим именно так, проста - эктор образовано от слова **action**, что в переводе означает *действие*. Дословный же перевод слова "эктор" - *действующее лицо* - слишком длинный и неудобный для употребления.
- Можно заметить промелькнувшее в определении эктора слово "прецедент". Что же это такое? Этот вопрос заинтересует нас еще больше, если вспомнить, что сейчас мы говорим о *диаграмме прецедентов*.

Диаграмма прецедентов (use case diagram)

- **Прецедент (use-case)** - описание отдельного аспекта поведения системы с точки зрения пользователя (Буч).

Определение вполне понятное и исчерпывающее, но его можно еще немного уточнить:

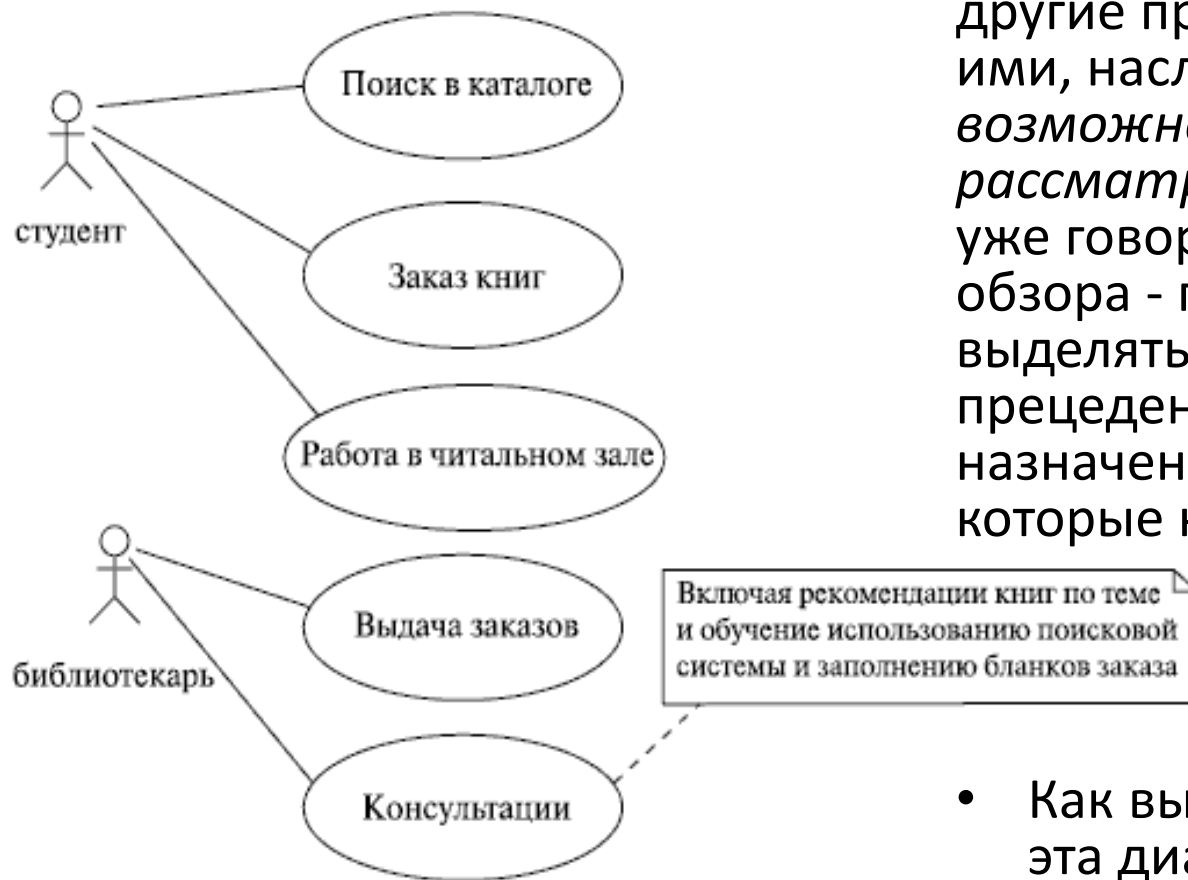
- **Прецедент (use case)** - описание множества последовательных событий (включая варианты), выполняемых системой, которые приводят к наблюдаемому эктором результату. Прецедент представляет поведение сущности, описывая взаимодействие между экторами и системой. Прецедент *не показывает*, "как" достигается некоторый результат, а только "что" именно выполняется.

Диаграмма прецедентов (use case diagram)

- Прецеденты обозначаются очень простым образом - в виде эллипса, внутри которого указано его название. *Прецеденты и акторы соединяются с помощью линий. Часто на одном из концов линии изображают стрелку, причем направлена она к тому, у кого запрашивают сервис, другими словами, чьими услугами пользуются. Это простое объяснение иллюстрирует понимание прецедентов как сервисов, пропагандируемое компанией IBM.*



Диаграмма прецедентов (use case diagram)



Прецеденты могут включать другие прецеденты, расширяться ими, наследоваться и т. д. *Все эти возможности мы здесь рассматривать не будем.* Как уже говорилось выше, цель этого обзора - просто научить читателя выделять диаграмму прецедентов, понимать ее назначение и смысл обозначений, которые на ней встречаются.

- Как вы думаете, что означает эта диаграмма?

Диаграмма прецедентов (use case diagram)

- Следует заметить, что иногда на диаграммах прецедентов *границы системы обозначают прямоугольником*, в верхней части которого может быть указано *название системы*. Таким образом, прецеденты - действия, выполняемые системой в ответ на действия эктора, - помещаются внутри прямоугольника.

Диаграмма прецедентов (use case diagram)

А вот еще один пример:

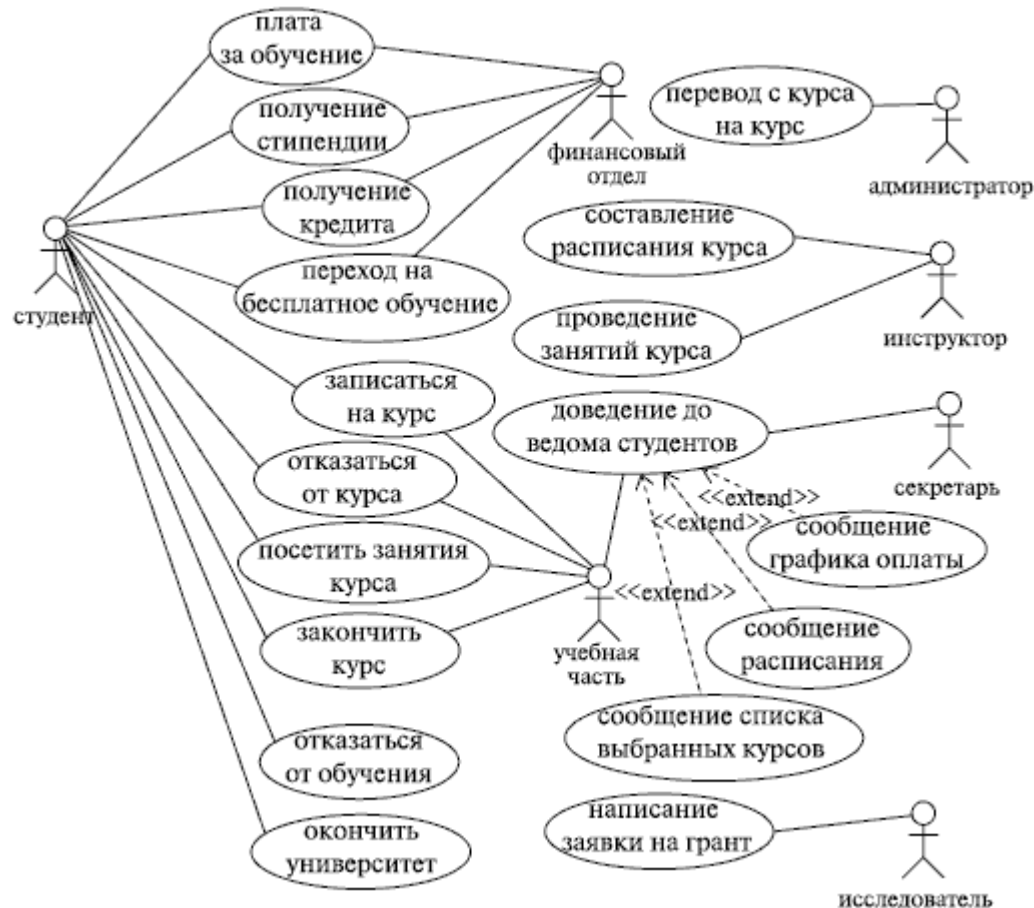


Диаграмма прецедентов (use case diagram)

Из всего сказанного выше становится понятно, что *диаграммы прецедентов* относятся к той группе диаграмм, которые представляют динамические или поведенческие аспекты системы.

Это отличное *средство для достижения взаимопонимания между разработчиками, экспертами и конечными пользователями* продукта.

Как мы уже могли убедиться, такие диаграммы очень просты для понимания и могут восприниматься и, что немаловажно, обсуждаться людьми, не являющимися специалистами в области разработки ПО.

Диаграмма прецедентов (use case diagram)

Подводя итоги, можно выделить такие **цели создания диаграмм прецедентов:**

- определение границы и контекста моделируемой предметной области на ранних этапах проектирования;
- формирование общих требований к поведению проектируемой системы;
- разработка концептуальной модели системы для ее последующей детализации;
- подготовка документации для взаимодействия с заказчиками и пользователями системы.

Диаграмма классов (class diagram)

- **Класс (class)** - категория вещей, которые имеют общие атрибуты и операции.

Классы - *это строительные блоки любой объектно-ориентированной системы*. Они представляют собой описание совокупности объектов с общими атрибутами, операциями, отношениями и семантикой.

При проектировании объектно-ориентированных систем диаграммы классов обязательны.

Классы используются в процессе анализа предметной области для составления *словаря предметной области* разрабатываемой системы. Это могут быть как абстрактные понятия предметной области, так и классы, на которые опирается разработка и которые описывают программные или аппаратные сущности.

Диаграмма классов (class diagram)

- **Диаграмма классов** - это набор статических, декларативных элементов модели.

Диаграммы классов могут применяться и при прямом проектировании, то есть в процессе разработки новой системы, и при *обратном проектировании* - описании существующих и используемых систем.

Информация с диаграммы классов напрямую отображается в исходный код приложения - в большинстве существующих инструментов UML-моделирования возможна *кодогенерация* для определенного языка программирования (обычно Java или C++). Таким образом, диаграмма классов - конечный результат проектирования и отправная точка процесса разработки.

Диаграмма классов (class diagram)

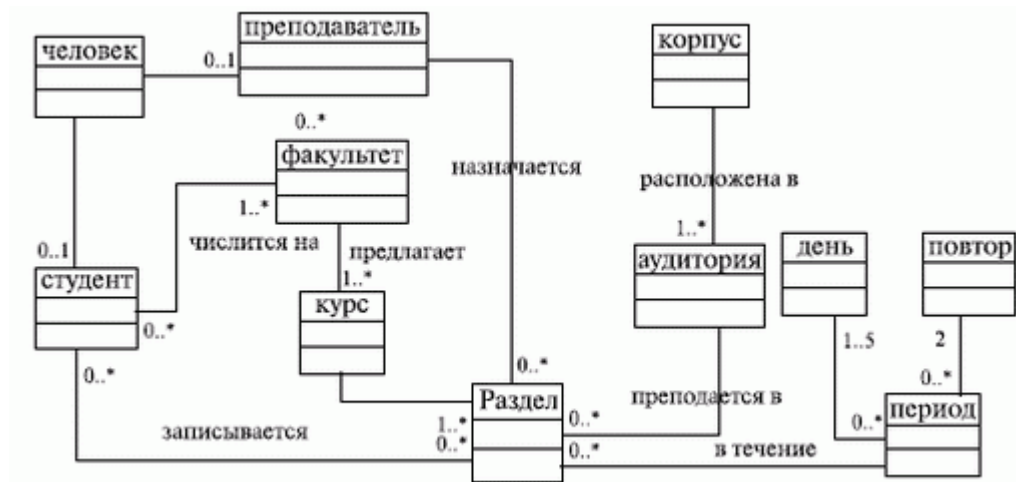
Итак, посмотрим на примеры диаграмм классов.



Пример весьма прост. Как видим, он, хоть и немного однобоко, иллюстрирует с помощью операции наследования или генерализации "генеалогическое древо" бытовой техники. Думаем, мы бы поняли смысл этой диаграммы, даже если бы ничего не знали о классах и не занимались программированием вообще.

Диаграмма классов (class diagram)

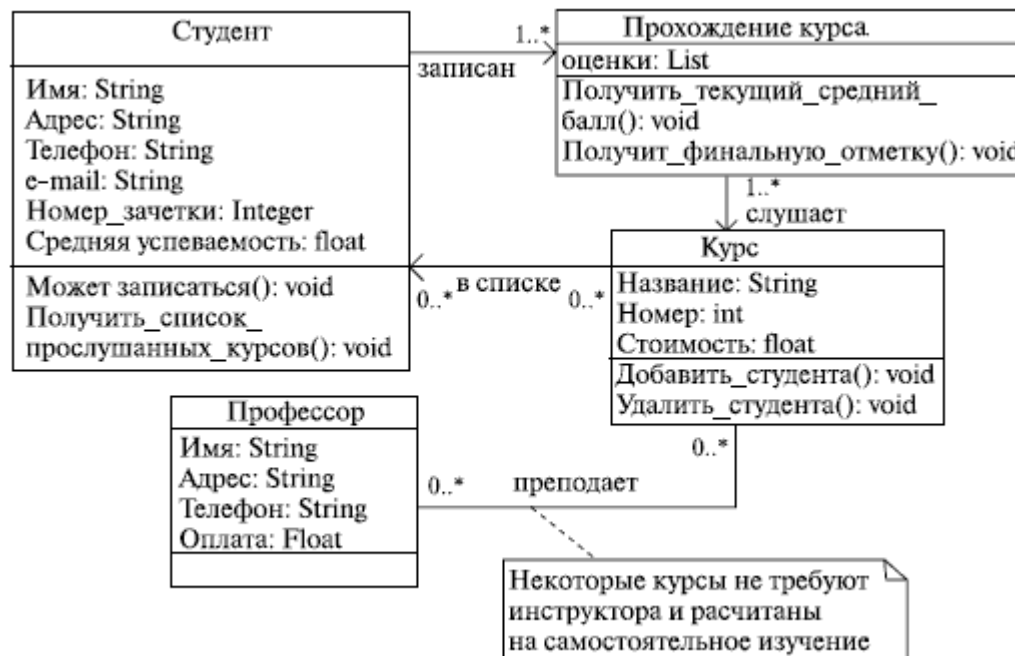
Рассмотрим еще пример:



И опять-таки смысл этой диаграммы ясен без особых пояснений. Даже бегло рассмотрев ее, можно легко догадаться, что она описывает предметную область задачи об автоматизации работы некоего вуза или учебного центра. Обратите внимание на обозначения кратности на концах связей.

Диаграмма классов (class diagram)

А теперь немного усложним задачу:



Как видим, здесь уже все более серьезно - кроме кратности обозначены свойства (и их типы) и операции, и вообще, эта диаграмма производит впечатление набора классов для реализации, а не просто описания предметной области, как предыдущие. Но, тем не менее, все равно все просто и понятно.

Диаграмма объектов (object diagram)

Объект (object) - экземпляр класса.

Объект (object) -

- конкретная материализация абстракции;
- сущность с хорошо определенными границами, в которой инкапсулированы состояние и поведение;
- экземпляр класса (вернее, классификатора - эктор, класс или интерфейс). Объект уникально идентифицируется значениями атрибутов, определяющими его состояние в данный момент времени.

Диаграмма объектов (object diagram)

"Второе" определение, по сути, просто расширяет "Бучевское". Да, действительно, объект - это экземпляр класса. Скажем, объектом класса "Микроволновая печь" из примера, приведенного выше, может быть и простейший прибор фирмы "Saturn" небольшой емкости и с механическим управлением, и навороченный агрегат с грилем, сенсорным управлением и системой трехмерного распределения энергии от Samsung или LG.

- Еще пример - все мы являемся объектами класса "человек" и различимы между собой по таким признакам (значениям атрибутов), как имя, цвет волос, глаз, рост, вес, возраст и т. д. (в зависимости от того, какую задачу мы рассматриваем и какие свойства человека для нас в ней важны).

Диаграмма объектов (object diagram)

- Объект, как и класс, обозначается прямоугольником, но его имя подчеркивается. Под словом имя здесь понимаем название объекта и наименование его класса, разделенные двоеточием. Для указания значений атрибутов объекта в его обозначении может быть предусмотрена специальная секция. Еще один нюанс состоит в том, что объект может быть анонимным: это нужно в том случае, если в данный момент не важно, какой именно объект данного класса принимает участие во взаимодействии.

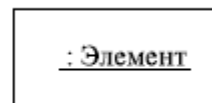
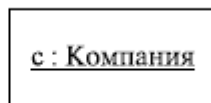
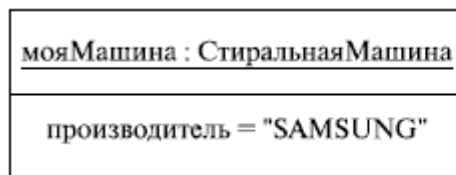


Диаграмма объектов (object diagram)

- *Диаграмма объектов - это своего рода снимок состояния системы в определенный момент времени, показывающий множество объектов, их состояния и отношения между ними в данный момент.*

Таким образом, *диаграммы объектов* представляют *статический вид системы с точки зрения проектирования и процессов*, являясь основой для сценариев, описываемых диаграммами взаимодействия. Говоря другими словами, *диаграмма объектов используется для пояснения и детализации диаграмм взаимодействия*, например, диаграмм последовательностей. Впрочем, авторам курса очень редко доводилось применять этот тип диаграмм.

Диаграмма объектов (object diagram)

Приведем простейший пример такой диаграммы:

О чем здесь идет речь, в принципе, понятно: некоторая фирма "раскручивает" новый товар или услугу. В этом процессе участвуют вице-президент по маркетингу, вице-президент по продажам, менеджер по продажам, торговый агент, специалист по рекламе, некое печатное издание и покупатель. Причем даже без указания сообщений, которыми обмениваются эти объекты, отлично видно, кто с кем взаимодействует. Кстати, обратите внимание, что на этой диаграмме все объекты анонимные!



Диаграмма объектов (object diagram)

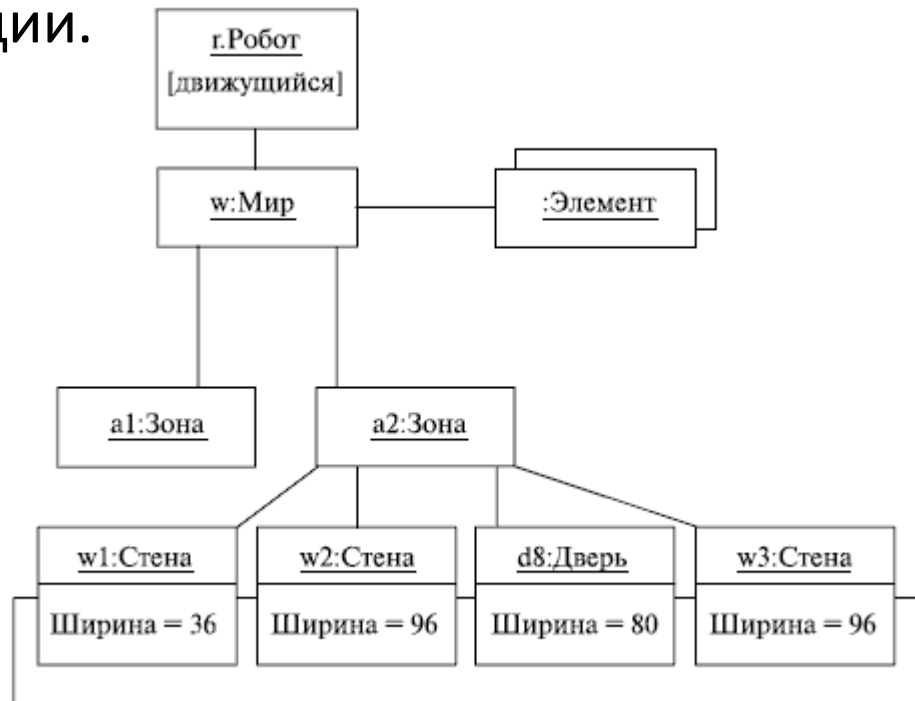
Другой пример:



Диаграмма объектов (object diagram)

И наконец, последний пример:

диаграмма объектов учебной среды "Робот" для Turbo Pascal, в которой наше поколение школьников училось основам алгоритмизации.



Спасибо за внимание!

Ищенко Алексей Петрович